



Proyecto Semana 06: API con Service Layer y Relaciones

N:M



Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]

⚠️ **IMPORTANTE:** Cada aprendiz trabaja sobre un dominio diferente.



Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que NO está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Entity A	Item	{YourEntityA}
Entity B	Tag	{YourEntityB}
N:M Relation	Items have many Tags	{your_nm_relation}



Objetivo

Construir una **API con arquitectura por capas** implementando:

- Service Layer para lógica de negocio
- Relaciones N:M con tabla intermedia
- Eager loading para optimización
- DTOs/Schemas separados por capa



Requisitos Funcionales (Adapta a tu Dominio)

Entities (Mínimo 3: 2 principales + tabla intermedia)

```
# Ejemplo genérico (Warehouse)
class Item(Base):
    __tablename__ = "items"
    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(unique=True)
    name: Mapped[str]

    # N:M relationship
    tags: Mapped[list["Tag"]] = relationship(
        secondary="item_tags", back_populates="items"
    )

class Tag(Base):
    __tablename__ = "tags"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(unique=True)
    color: Mapped[str]

    # N:M inverse
    items: Mapped[list["Item"]] = relationship(
```

```

        secondary="item_tags", back_populates="tags"
    )

# Association table
item_tags = Table(
    "item_tags", Base.metadata,
    Column("item_id", ForeignKey("items.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True),
    Column("assigned_at", DateTime, default=datetime.utcnow),
)

```

Service Layer

```

# Ejemplo genérico
class ItemService:
    def __init__(self, db: Session):
        self.db = db

    async def add_tag_to_item(self, item_id: int, tag_id: int) -> Item:
        """Agrega un tag a un item (lógica de negocio)"""
        item = await self._get_item(item_id)
        tag = await self._get_tag(tag_id)

        if tag in item.tags:
            raise TagAlreadyAssignedError(item_id, tag_id)

        item.tags.append(tag)
        await self.db.commit()
        return item

```

Endpoints

Método	Endpoint	Descripción
POST	/items/{id}/tags/{tag_id}	Agregar tag a item
DELETE	/items/{id}/tags/{tag_id}	Quitar tag de item
GET	/items/{id}/tags	Listar tags de un item
GET	/tags/{id}/items	Listar items con un tag



Estructura del Proyecto

```

starter/
├─ main.py
├─ database.py
├─ models/
│   ├─ item.py
│   ├─ tag.py
│   └─ associations.py
├─ schemas/
├─ services/
│   └─ item_service.py

```

```
| └─ tag_service.py
| └─ routers/
| └─ pyproject.toml
| └─ Dockerfile
└─ docker-compose.yml
```

✓ Criterios de Evaluación

Criterio	Puntos
Funcionalidad (40%)	
Relación N:M funciona	15
Service Layer implementado	15
Endpoints de asociación	10
Adaptación al Dominio (35%)	
Relación N:M coherente	12
Lógica de negocio en services	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Separación de capas clara	10
Eager loading correcto	10
Código limpio	5
Total	100

⚠ Política Anticopia

- ✗ **No copies** el ejemplo genérico "Item/Tag"
- ✓ **Diseña** una relación N:M de tu dominio
- ✓ **Implementa** lógica de negocio real

📚 Recursos

- [SQLAlchemy Many-to-Many](#)
- [Pool de Dominios](#)

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)